

Java Tutorial

Last Updated : 8 Jul, 2026

-
-
-

Java is a high-level, object-oriented programming language used to build web apps, mobile applications, and enterprise software systems.

- Java is a platform-independent language, which means code written in Java can run on any device that supports the Java Virtual Machine (JVM).
- Syntax and structure are similar to C-based languages like C++ and C#.

Hello World Program

The following is a simple program that displays the message “Hello, World!” on the screen.

```
public class Geeks{  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

Output

Hello World!

Explanation: Above program defines a class Geeks with a main method that serves as the entry point and prints "Hello World!" to the console.

Why Learn Java?

- Used to build Android apps, desktop and web apps, enterprise backend systems, and cloud-based software.
- In high demand with many job opportunities in software development.
- Has popular frameworks like Spring and Hibernate which makes it powerful for enterprise applications.
- Supports object-oriented programming for clean, maintainable and reusable code.
- Top companies like Amazon, Netflix, and LinkedIn use Java.

Basics

Understand core Java concepts such as syntax, variables, data types, loops, and decision-making to start your programming journey.

- [Introduction](#)
- [Download and Install Java](#)
- Download and Install [IntelliJ IDEA](#), [Eclipse](#)
- [Creating First Java Application in IntelliJ IDEA](#)
- [JDK vs JRE vs JVM](#)

- [Print output](#)
- [Taking Input](#)
- [Identifiers](#)
- [Keywords](#)
- [Variables](#)
- [Data Types](#)
- [Literals](#)
- [Wrapper Classes](#)
- [Operators](#)
- [Decision Making](#)
- [Loops and Jump Statements](#)
- **Project:** [Number Guessing Game](#)

Methods

Java methods are reusable blocks of code that perform specific tasks and help organize your program. They improve code readability, reduce repetition, and make debugging easier.

- [Introduction](#)
- [Static Methods vs Instance Methods](#)
- [Access Modifiers](#)
- [Command Line Arguments](#)
- [Variable Arguments \(Varargs\)](#)

Arrays

Java arrays are containers that store multiple values of the same data type in a single variable. They provide an efficient way to manage and access collections of data using index-based positions.

- [Introduction](#)

- [Multi-Dimensional Arrays](#)
- [Jagged Arrays](#)
- [Arrays Class](#)
- [Final Arrays](#)
- **Projects:** [Tic-Tac-Toe Game](#)

Strings

Java Strings represent sequences of characters and are widely used in text processing. They are immutable, meaning once created, their values cannot be changed.

- [Introduction](#)
- [Why Strings are Immutable](#)
- [.equals\(\) vs ==](#)
- [String Concatenation](#)
- [String Methods](#)
- [String Class](#)
- [StringBuffer Class](#)
- [StringBuilder Class](#)
- [Strings vs StringBuffer vs StringBuilder](#)

OOP Concepts

Java follows the Object-Oriented Programming (OOP) paradigm, which organizes code into classes and objects. Core OOP principles like inheritance, encapsulation, polymorphism, and abstraction make Java modular and scalable.

- [Introduction](#)
- [Classes and Objects](#)
- [Constructors](#)
- [this Keyword](#)
- [super Keyword](#)
- [Encapsulation](#)
- [Inheritance](#)
- [Polymorphism](#)
- [Abstraction](#)
- [Object Class](#)
- [Packages](#)

- **Project:** [Simple Banking Application](#)

Interfaces

Java interfaces define a contract that classes must follow, specifying method signatures without implementations. They enable abstraction and support multiple inheritance in Java through a clean, structured approach.

- [Interfaces](#)
- [Class vs Interface](#)
- [Functional Interface](#)
- [Nested Interface](#)
- [Marker Interface](#)
- **Project:** [Employee Management System](#)

Exception Handling

Java Exception Handling is a mechanism to handle runtime errors, ensuring the program runs smoothly without crashing. It uses keywords like try, catch, throw, throws, and finally to manage exceptions.

- [Introduction](#)
- [Try-Catch Block](#)
- [Final, Finally and Finalize](#)
- [Throw and Throws](#)
- [Customized Exception Handling](#)
- [Chained Exceptions](#)
- [Null Pointer Exceptions](#)
- [Exception Handling with Method Overriding](#)

Regex

Java Regex (Regular Expressions) allows pattern matching and text manipulation using the java.util.regex package. It is powerful for validating, searching, and replacing strings based on specific patterns.

- [Introduction](#)
- [Matcher Class](#)
- [Character Class](#)
- [Quantifiers](#)

Memory Allocation

Java Memory Allocation refers to how memory is assigned to variables, objects, and classes during program execution. It involves stack and heap memory, with the JVM managing allocation and garbage collection automatically.

- [Java Memory Management](#)
- [How Java Objects Stored in Memory?](#)
- [Types of Memory Areas Allocated by JVM](#)
- [Stack vs Heap Memory Allocation](#)
- [Garbage Collection](#)
- [Types of JVM Garbage Collectors](#)
- [Memory Leaks](#)

Generics

Generics provide a way to create classes, interfaces, and methods that work with different data types while ensuring type safety and reducing the need for explicit type casting.

- [Generic Classes](#)
- [Wildcards](#)
- [Bounded Types](#)
- [Type Erasure](#)

Collections

Java Collections provide a framework for storing and manipulating groups of objects efficiently. It includes interfaces like List, Set, and Map, along with classes like ArrayList, HashSet, and HashMap.

- [Collections Class](#)
- [Collection Interface](#)
- [List Interface](#)
- [ArrayList](#)
- [LinkedList](#)
- [Set Interface](#)
- [HashSet](#)
- [Linked HashSet](#)
- [TreeSet](#)
- [Queue Interface](#)
- [Priority Queue](#)
- [Deque Interface](#)

- [Map Interface](#)
- [HashMap](#)
- [Iterator](#)
- [Comparator Interface](#)
- [Comparable Interface](#)
- [Collection Framework Complete Tutorial](#)
- **Project:** [Face Detection System](#)

Java 8+ Features

Java 8 and later versions introduced powerful features such as Lambda Expressions, Stream API, Functional Interfaces, Optional, the Date and Time API, and other enhancements to make code more concise, readable, and efficient.

- [Lambda Expressions](#)
- [Predicate](#)
- [Consumer](#)
- [Supplier](#)
- [Method References](#)
- [Streams](#)
- [Optional](#)
- [Collectors](#)

Date and Time API

The Java Date & Time API provides modern, immutable classes for handling dates, times, time zones, formatting, and date-time calculations efficiently and accurately.

- [Introduction](#)
- [LocalDate](#)
- [LocalTime](#)
- [LocalDateTime](#)
- [Duration](#)
- [Period](#)
- [DateTimeFormatter](#)

Multithreading and Synchronization

Java Multithreading allows concurrent execution of two or more threads, enabling efficient CPU utilization and faster program performance. It is commonly used for tasks that required parallel processing and responsiveness from multiple ends.

- [Introduction](#)
- [Threads](#)
- [Thread.start\(\) vs Thread.run\(\) Method](#)
- [Thread.sleep\(\) Method](#)
- [Runnable Interface](#)
- [Main Thread](#)
- [Thread Priority in Multithreading](#)
- [Daemon Thread](#)
- [Java Synchronization](#)
- [Thread Safety](#)
- [Locks in Java](#)
- [Lock vs Monitor in Concurrency](#)
- [Lock Framework vs Thread Synchronization](#)
- [Reentrant Lock](#)
- [Deadlock in Multithreading](#)
- [Thread Pools](#)
- [Executor Framework](#)
- [Multithreading Complete Tutorial](#)
- **Project:** [Snake Game](#)

File Handling

Java File Handling enables programs to create, read, write, and manipulate files stored on the system. It uses classes from the java.io and java.nio packages for efficient file operations.

- [Introduction to Java IO](#)
- [Reader Class](#)
- [Writer Class](#)
- [File Handling](#)
- [File Class](#)
- [FileInputStream](#)
- [FileOutputStream](#)
- [FileReader Class](#)
- [FileWriter Class](#)

- [FileOutput Stream](#)
- [BufferedReader Input Stream](#)
- [BufferedReader Output stream](#)
- [FilePermission Class](#)
- [FileDescriptor Class](#)
- **Project:** [Text Editor](#)

Networking

Java Networking enables communication between devices over a network using classes from the java.net package. It supports protocols like TCP and UDP for building client-server applications and data exchange.

- [Introduction](#)
- [Socket Programming](#)
- [Server Socket Class](#)
- [URL Class and Methods](#)
- **Project:** [Chat Application](#)

Java Database Connectivity(JDBC)

Java Database Connectivity (JDBC) enables Java applications to connect and interact with databases for storing and retrieving data efficiently.

- [Introduction](#)
- [JDBC Driver](#)
- [JDBC Connection](#)
- [Types of Statements in JDBC](#)
- [Transactions](#)
- [ResultSet](#)
- [Metadata](#)
- [Connection Pooling](#)

Quiz and Coding Practice

Practice chapter-wise Java quizzes and solve coding problems to improve your programming knowledge, logical thinking, and problem-solving skills.

- [Java Topic-wise Quiz](#)
- [Coding Practice Problems](#)

Interview Questions

This section provides a collection of commonly asked Java interview questions and answers to help you prepare for technical interviews.

- [Core Java Interview Questions and Answers](#)
- [Advanced Java Interview Questions and Answers](#)

Important Links

- [Comparison of various programming languages](#)
- [Companies That Use Java](#)
- [Careers and Jobs in Java](#)